

Modül 5

Signals & Reactivity

Derinlemesine

Modül:	5 / 30
Ders Sayısı:	8 ders
Seviye:	Orta-İleri
Versiyon:	Angular v21

Hazırlayan: Claude • Türkçe Angular v21 Eğitimi

📦 Modül 5'e Hoş Geldin

Bu modül, Angular'ın kalbi olan signals sistemini derinlemesine işliyor. Modül 4'te yüzeysel olarak gördüğümüz `signal()`, `computed()`, `effect()`'i şimdi her detayıyla öğreneceğiz. Ayrıca yeni API'lar: `linkedSignal()`, `resource()`, `untracked()`, `toSignal()` ve RxJS interop.

Bu modülün sonunda yapabileceklerin:

- ✓ Signal sisteminin nasıl çalıştığını derinlemesine anlatabilmek
- ✓ Glitch-free reactivity'nin neden önemli olduğunu açıklayabilmek
- ✓ `signal()`, `computed()`, `effect()` arasındaki farkları net bilmek
- ✓ `untracked()` ne zaman gerekli, kullanabilmek
- ✓ `linkedSignal()` ile state senkronizasyonu yapabilmek
- ✓ `resource()` ile async data yönetimi
- ✓ RxJS Observable'ları signal'a çevirip tersini yapabilmek
- ✓ Production-grade signal patterns ile kompleks state yönetimi

📦 Önemli Modül

Bu modül Angular'ın en kritik modüllerinden biri. Buradaki kavramlar tüm sonraki modüllerin temeli. Acele etme — her dersi kavradığından emin ol, gerekirse iki kere oku.

Signal Nedir, Neden Var?

Önce "signal" kavramının ne anlama geldiğini ve Angular'a neden eklendiğini anlamamız gerek. Bu temel olmadan diğer her şey havada kalır.

□ Reactive Programming Nedir?

Reactive programming: Veriler değiştiğinde, ona bağlı her şey otomatik güncellenir. Sen her şeyi manuel yönetmek zorunda değilsin — sistem, bağımlılıkları takip eder.

Klasik (imperative) yaklaşım:

```
let firstName = 'John';
let lastName = 'Doe';
let fullName = firstName + ' ' + lastName; // 'John Doe'

firstName = 'Jane';
console.log(fullName); // Hala 'John Doe' – manual güncellenmedi!
```

Reactive yaklaşım:

```
const firstName = signal('John');
const lastName = signal('Doe');
const fullName = computed(() => firstName() + ' ' + lastName());

console.log(fullName()); // 'John Doe'
firstName.set('Jane');
console.log(fullName()); // 'Jane Doe' – OTOMATİK güncellendi!
```

□ Tarihsel Bağlam — Angular'da Reactivity'nin Evrimi

Faz 1: Zone.js (Angular 2-15)

Eski Angular, Zone.js denilen bir kütüphane kullanırdı. Zone.js, JavaScript'in tüm async operasyonlarını (setTimeout, Promise, click, vs.) "patch" ederek izlerdi. Herhangi bir async olayda Angular tüm component tree'yi tarar, neyin değiştiğini bulurdu.

Problemler:

- ✗ Performans: Tüm tree taraması yavaş
- ✗ Bundle size: Zone.js büyük (~100kb)
- ✗ Magic: Geliştirici neyin neden tetiklendiğini anlamıyor
- ✗ Debug zor: "Bu component neden re-render oldu?" — bilmiyorsun
- ✗ RxJS gerekli: Manuel reactivity için RxJS öğrenmek zorundaydın

Faz 2: Signals (Angular 16+)

Angular ekibi SolidJS, Vue 3, Svelte 5 gibi framework'lerden ilham alarak signal-based reactivity sistemini ekledi. Signal'lar fine-grained reactivity sağlar — sadece gerçekten değişen kısım güncellenir.

Avantajlar:

- ✓ Performans: Sadece bağımlılıkları tarar, %90+ hızlı
- ✓ Bundle size: Zone.js kaldırılabilir (zoneless)
- ✓ Açık: Geliştirici bağımlılıkları tam görür
- ✓ Debug kolay: Signal değişimi izlenebilir
- ✓ RxJS opsiyonel: Basit reactivity için signal yeter

□ Signal Nedir — Mental Model

Signal: Değeri olan ve değiştiğinde dinleyicileri haberdar eden bir kutu. JavaScript variable değil, daha akıllı bir şey.

```
const count = signal(0);

count();           // OKUR (değeri döner)
count.set(5);      // YAZAR (değeri değiştirir)
count.update(c => c + 1); // UPDATE (mevcut değeri kullanarak değiştirir)
```

Üç ana operasyon: read, set, update. Variable değil çünkü her okuma/yazma tracked — sistem kim okuyor, kim yazıyor biliyor.

□ Signal Üç Türü Vardır

Angular'da üç tür signal var. Hepsini birkaç ders boyunca derinlemesine işleyeceğiz, ama önce genel bakış:

Tür	Ne İşe Yarar?	Örnek
signal()	Yazılabilir kaynak (state)	count, name, todos
computed()	Türetilmiş değer (derived state)	fullName, filteredItems, total
effect()	Yan etki (side effect)	localStorage sync, logging, analytics

□ Mental Model

signal() = "Bu bir state, değişebilir"
computed() = "Bu bir hesap, başka signal'lardan otomatik türer"
effect() = "Bu bir aksiyon, signal'lar değişince çalışır"

□ İlk Pratik

```
import { signal, computed, effect } from '@angular/core';

@Component({...})
export class CounterComponent {
  // STATE (yazılabilir kaynak)
  count = signal(0);

  // DERIVED STATE (otomatik hesaplanır)
  doubled = computed(() => this.count() * 2);
  isPositive = computed(() => this.count() > 0);
  message = computed(() => `${this.count()} (x2: ${this.doubled()})`);

  // SIDE EFFECT (signal değişince çalışır)
  constructor() {
    effect(() => {
      console.log('Count değişti:', this.count());
      // localStorage, document.title, analytics, vs.
    });
  }

  // ACTIONS
  increment() { this.count.update(c => c + 1); }
  decrement() { this.count.update(c => c - 1); }
  reset() { this.count.set(0); }
}
```

Bu kodda üç signal türünü de görüyorsun. Sonraki derslerde her birini detaylı işleyeceğiz.

□ Bu Dersten Çıkarılacaklar

- ✓ Reactive programming'in ne olduğunu biliyorum
- ✓ Angular'da Zone.js'den signals'a geçişin sebebini anlıyorum
- ✓ Signal'ın "akıllı kutu" mental modelini kavradım
- ✓ Üç signal türünü (signal, computed, effect) genel olarak biliyorum
- ✓ Her birinin nerede kullanıldığını tahmin edebilirim

signal() Derinlemesine

Şimdi yazılabilir signal'ları detaylı inceleyelim. API'sı, options'ları, best practice'leri, anti-pattern'leri.

□ Temel API

```
import { signal, WritableSignal } from '@angular/core';

// Type inference ile
const count = signal(0);           // WritableSignal<number>
const name = signal('John');       // WritableSignal<string>
const items = signal<string[]>([]); // WritableSignal<string[]> (explicit)

// Okuma (fonksiyon çağrısı)
count(); // 0

// Yazma
count.set(5);           // Direkt değer
count.update(c => c + 1); // Mevcut değer üzerinden

// Asla:
count = 5; // □ Signal'ı override edemezsin (reactivity kaybolur)
```

⚙️ Signal Options

```
import { signal } from '@angular/core';

// equal – Custom equality function
const user = signal({ id: 1, name: 'John' }, {
  equal: (a, b) => a.id === b.id // Sadece id'ye bak
});

// Eğer aynı id ile yeni object set edersen, signal değişmemiş sayılır
user.set({ id: 1, name: 'Jane' }); // Aynı sayılır, dinleyiciler tetiklenmez

// Default: ===
const count = signal(0);
count.set(0); // Aynı, tetiklenmez

// Object/array için referans karşılaştırma
const items = signal([1, 2, 3]);
items.set([1, 2, 3]); // Yeni referans, tetiklenir!

// Önemli: Object/array için MUTATION ETME
const obj = signal({ count: 0 });
obj().count = 5; // ❌ Reactivity yok! Set ile yenisini ver:
obj.set({ count: 5 }); // ✓
obj.update(o => ({ ...o, count: 5 })); // ✓ Spread ile
```

⚠ Mutation vs Immutable Update

Signal'lar immutable update bekler. Object/array'ı mutate ettiğinde signal değişimi algılayamaz. Her zaman yeni object/array oluştur.

📖 WritableSignal API

Method	Ne Yapar?	Örnek
()	Değeri okur (tracking)	count()
.set(value)	Direkt değer atar	count.set(5)
.update(fn)	Mevcut değeri dönüştürür	count.update(c => c+1)
.asReadonly()	Read-only versiyon döner	public x = this._x.asReadonly()

📖 asReadonly Pattern (Encapsulation)

Service'lerde private writable + public readonly pattern'i kullanılır. Dış component'ler sadece okuyabilir, sadece service kendi metodları ile değiştirebilir.

```

@Injectables({ providedIn: 'root' })
export class TodoService {
  // PRIVATE – sadece bu class değiştirebilir
  private _todos = signal<Todo[]>([]);

  // PUBLIC – dışarıdan sadece okunabilir
  readonly todos = this._todos.asReadonly();
  //          ^^^^^^^^^^^^^
  //          set/update yok, sadece okuma

  // Action methods – kontrollü değişim
  add(todo: Todo): void {
    this._todos.update(todos => [...todos, todo]);
  }

  remove(id: string): void {
    this._todos.update(todos => todos.filter(t => t.id !== id));
  }
}

// Component'te:
const todoService = inject(TodoService);

todoService.todos();           // ✓ Okuma OK
todoService.todos.set([]);     // ✗ Compile error (asReadonly)
todoService.remove('123');     // ✓ Action ile değişim

```

❑ Anti-Patterns

- ✗ Variable gibi davranma: `count = 5` yazma, `count.set(5)` yaz
- ✗ Mutate etme: `items().push(x)` yapma, `items.update(arr => [...arr, x])` yap
- ✗ Her şey için signal: Constant değerler için signal gereksiz, plain property yeter
- ✗ Çok sık encapsulation: Service'in writable signal'ını direkt expose etme
- ✗ Parantezsiz okuma: `count` signal referansı, `count()` değeri

❑ Pratik Senaryolar

Senaryo 1: Counter

```

counter = signal(0);
increment() { this.counter.update(c => c + 1); }
decrement() { this.counter.update(c => c - 1); }
reset() { this.counter.set(0); }

```

Senaryo 2: Form Input

```
title = signal('');

onInput(event: Event) {
  const target = event.target as HTMLInputElement;
  this.title.set(target.value);
}
```

Senaryo 3: Toggle

```
isOpen = signal(false);
toggle() { this.isOpen.update(v => !v); }
```

Senaryo 4: Object State

```
user = signal({ name: '', email: '', age: 0 });

updateName(name: string) {
  this.user.update(u => ({ ...u, name })); // Spread!
}

updateMultiple(changes: Partial<User>) {
  this.user.update(u => ({ ...u, ...changes }));
}
```

Senaryo 5: Array State

```
items = signal<Item[]>([]);

add(item: Item) {
  this.items.update(arr => [...arr, item]);
}

remove(id: string) {
  this.items.update(arr => arr.filter(i => i.id !== id));
}

updateAt(id: string, changes: Partial<Item>) {
  this.items.update(arr =>
    arr.map(i => i.id === id ? { ...i, ...changes } : i)
  );
}
```

☐ Bu Dersten Çıkarılacaklar

- ✓ signal() oluşturup okuyabiliyorum
- ✓ set(), update() farkını biliyorum
- ✓ equal option'ını ne zaman kullanacağımı anlıyorum
- ✓ asReadonly() ile encapsulation yapabiliyorum
- ✓ Immutable update pattern'ini içselleştirdim
- ✓ Object/array için spread kullanmayı biliyorum

computed() Derinlemesine

Türetilmiş değerler için `computed()`. Cache, dependency tracking, glitch-free reactivity — hepsini anlayacağız.

□ Temel Kullanım

```
import { signal, computed } from '@angular/core';

const firstName = signal('John');
const lastName = signal('Doe');

// Computed – diğer signal'lardan türer
const fullName = computed(() => firstName() + ' ' + lastName());

console.log(fullName()); // 'John Doe'

firstName.set('Jane');
console.log(fullName()); // 'Jane Doe' (otomatik!)
```

□ Computed'in Sihiri — Üç Özellik

1. Otomatik Dependency Tracking

Computed, içinde hangi signal'ları okuduğunu Angular'a söylemez. Angular otomatik takip eder. Sen sadece read et — bağımlılıklar otomatik kayıt edilir.

```
const a = signal(1);
const b = signal(2);
const c = signal(3);

const sum = computed(() => a() + b());
// Bağımlılıklar: a, b
// c okunmadı, c değişse sum yeniden hesaplanmaz
```

2. Lazy Evaluation

Computed çağrılmadığı sürece hesaplanmaz. İlk okumada hesaplanır, sonra cache'lenir.

```
const expensive = computed(() => {
  console.log('Calculating...');
  return /* heavy computation */;
});

// Henüz hesaplanmadı (console'da çıktı yok)

expensive(); // İlk çağrı – hesaplanır, console'da "Calculating..."
expensive(); // İkinci çağrı – CACHE'ten döner, yeniden hesaplanmaz
```

3. Memoization (Cache)

Bağımlılıklar değişmedikçe, computed cache'den döner. Her okumada yeniden hesaplama yok.

```
const items = signal([1, 2, 3, 4, 5]);
const total = computed(() => {
  console.log('Hesaplanıyor...');
  return items().reduce((sum, n) => sum + n, 0);
});

total(); // "Hesaplanıyor..." → 15
total(); // (sessiz) → 15 (cache)
total(); // (sessiz) → 15 (cache)

items.set([10, 20]);
total(); // "Hesaplanıyor..." → 30 (yeniden)
```

⚡ Glitch-Free Reactivity

Bu Angular'ın özel özelliği. Bir signal değişimi diğer signal'ları zincirleme etkilese bile, tüm bağımlı computed'lar tutarlı durumda kalır.

```
const x = signal(1);
const y = computed(() => x() * 2);
const z = computed(() => x() + y());

x.set(5);
// y şimdi 10, z şimdi 15 – ASLA aralarında bir tutarsızlık yaşanmaz
```

Diğer framework'lerde sorun: x değişince y güncellenmeden z hesaplanırsa, z yanlış değerle çalışabilir. Angular bunu garantiler — topological sort ile.

📦 Pratik Örnekler

Örnek 1: Filter

```
items = signal<Item[]>([...]);
filter = signal<string>('');

filteredItems = computed(() => {
  const term = this.filter().toLowerCase();
  if (term === '') return this.items();
  return this.items().filter(i => i.name.toLowerCase().includes(term));
});
```

Örnek 2: Stats

```
todos = signal<Todo[]>([]);

stats = computed(() => {
  const all = this.todos();
  return {
    total: all.length,
    active: all.filter(t => !t.completed).length,
    completed: all.filter(t => t.completed).length,
  };
});
```

Örnek 3: Validation

```
email = signal('');
password = signal('');

emailValid = computed(() => /\S+@\S+\.\S+/.test(this.email()));
passwordValid = computed(() => this.password().length >= 8);
formValid = computed(() => this.emailValid() && this.passwordValid());
```

Örnek 4: Chained Computed

```
cart = signal<CartItem[]>([]);

subtotal = computed(() =>
  this.cart().reduce((sum, item) => sum + item.price * item.quantity, 0)
);

tax = computed(() => this.subtotal() * 0.18); // %18 KDV
total = computed(() => this.subtotal() + this.tax());
formattedTotal = computed(() => `₺${this.total().toFixed(2)}`);
```

❏ Anti-Patterns

- ✗ Side effect: Computed içinde `localStorage.setItem()` gibi side effect yapma — bu effect() işi
- ✗ Signal set: Computed içinde `otherSignal.set()` yapma — sonsuz döngü
- ✗ Random/Date: `Math.random()` veya `new Date()` kullanma — pure function olmalı, aynı input için aynı output
- ✗ HTTP isteği: Computed içinde `fetch` yapma — `resource()` kullan (M5.7)

✗ Tek seferlik computed: Sadece bir kere kullanılacaksa method olabilir

□ **computed() vs method**

Bazen "method da yazabilirim, niye computed?" diye düşünebilirsin. Fark çok kritik:

Konu	computed()	method
Cache	✓ Evet, memoize	✗ Her çağrıda yeniden
Reactivity	✓ Otomatik track	✗ Yok
Performance	✓ Optimum	Pahalı işlemler tekrarlanır
Template kullanımı	✓ Verimli	Her render'da yeniden

Template'te kullanılacaksa her zaman computed. Çünkü template her render'da çağrılır, method olsa sürekli yeniden hesaplanır.

□ **Bu Dersten Çıkarılacaklar**

- ✓ computed() oluşturup kullanabiliyorum
- ✓ Dependency tracking'in nasıl çalıştığını anlıyorum
- ✓ Lazy evaluation ve memoization avantajlarını biliyorum
- ✓ Glitch-free reactivity'nin önemini kavradım
- ✓ computed vs method farkını net biliyorum
- ✓ Chained computed'lar yazabiliyorum
- ✓ Side effect, mutation, async yapmaması gerektiğini biliyorum

effect() Derinlemesine

Side effect'leri yönetmek için `effect()`. Ne zaman tetiklenir, ne zaman kullanmalı, ne zaman kullanmamalı.

□ effect() Nedir?

`effect()`: Bağımlı olduğu signal'lar değiştiğinde çalışan bir side effect. "Bu signal'leri izle, değişince şunu yap" şeklinde düşün.

```
import { Component, signal, effect } from '@angular/core';

@Component({...})
export class MyComponent {
  count = signal(0);

  constructor() {
    effect(() => {
      console.log('count değişti:', this.count());
      document.title = `Count: ${this.count()}`;
    });
  }
}
```

□ Effect Ne Zaman Tetiklenir? — KURALLAR

Bu Modül 4'te de tartıştık ama tekrar — en yaygın kafa karışıklığı buradadır.

Kural 1: İlk Çalışma

Effect mount edildiğinde bir kez otomatik çalışır.

```
constructor() {
  effect(() => {
    console.log('Hello!');
    // Component yaratıldığında BİR KEZ çalışır
  });
}
```

Kural 2: Sadece İçeride OKUNAN Signal'leri İzler

```

count = signal(0);
name = signal('John');
email = signal('a@b.c');

constructor() {
  effect(() => {
    console.log(this.count()); // OKUNDU → izleniyor
    console.log(this.name());  // OKUNDU → izleniyor
    // email OKUNMADI
  });
}

this.count.set(5); // ✓ Effect tetiklenir
this.name.set('Jane'); // ✓ Effect tetiklenir
this.email.set('x'); // ✗ TETİKLENMEZ

```

Kural 3: Dynamic Dependencies

Effect her çalıştığı anda bağımlılıklar yeniden hesaplanır.

```

isActive = signal(true);
userName = signal('John');
userAge = signal(30);

constructor() {
  effect(() => {
    if (this.isActive()) {
      console.log(this.userName()); // isActive=true iken bağımlı
    } else {
      console.log(this.userAge());  // isActive=false iken bağımlı
    }
  });
}

// isActive=true → bağımlılıklar: isActive, userName
// isActive=false → bağımlılıklar: isActive, userAge

```

Kural 4: Microtask Batching

Aynı tick'te birden fazla signal değişirse, effect tek seferde çalışır.

```
count = signal(0);

constructor() {
  effect(() => {
    console.log('Effect:', this.count());
  });
}

// Method içinde:
this.count.set(5);
this.count.set(10);
this.count.set(15);
// Effect henüz çalışmadı

// Microtask:
// → Effect bir kez çalışır, count() === 15 (sadece son değer)
```

Kural 5: Cleanup

Component yok edildiğinde effect otomatik durur.

```
constructor() {
  effect((onCleanup) => {
    const interval = setInterval(() => { /* ... */ }, 1000);

    // Her re-run'da ve component destroy'da çalışır
    onCleanup(() => {
      clearInterval(interval);
    });
  });
}
```

□ Effect'i Ne Zaman Kullanmalı?

Use Case 1: localStorage Sync

```
constructor() {
  effect(() => {
    localStorage.setItem('todos', JSON.stringify(this.todos()));
  });
}
// todos değişince otomatik kaydedilir
```

Use Case 2: Document Title

```
constructor() {
  effect(() => {
    document.title = `${this.appName()} - ${this.userName()}`;
  });
}
```

Use Case 3: Analytics/Logging

```
constructor() {
  effect(() => {
    const route = this.currentRoute();
    if (route) {
      gtag('event', 'page_view', { page: route });
    }
  });
}
```

Use Case 4: 3rd Party Library Sync

```
constructor() {
  effect(() => {
    const theme = this.theme();
    this.chart?.update({ theme });
  });
}
```

Use Case 5: Debug

```
constructor() {
  effect(() => {
    console.log('User state:', this.userState());
  });
}
```

❑ Effect'i Ne Zaman KULLANMA?

⚠ Anti-Pattern 1: Computed Olabilecek Şey

Kötü:

```
effect(() => this.derived.set(this.count() * 2));
```

İyi:

```
derived = computed(() => this.count() * 2);
```

⚠ Anti-Pattern 2: HTTP Request

Kötü:

```
effect(() => { this.http.get(...).subscribe(...); });
```

İyi: resource() veya httpResource() kullan (M5.7)

⚠ Anti-Pattern 3: Signal'i Effect İçinde Set Etmek

Angular varsayılan olarak engeller. allowSignalWrites ile zorlayabilirsin ama bu sonsuz döngü riski. Çoğu durumda computed() istediğin şeyi yapar.

❑ Effect Options

```
effect(() => { ... }, {
  manualCleanup: false,    // true → manuel cleanup gerekli
  injector: customInjector, // Component dışında effect oluşturmak için
});

// Manuel cleanup örneği (Service'te):
@InjectableView()
export class MyService {
  constructor() {
    const ref = effect(() => {
      // ...
    });

    // Sonra durdurmak istersen:
    // ref.destroy();
  }
}
```

❑ Karar Ağacı

❑ Hangisini Kullanmalıyım?

İhtiyacın türemiş değer mi? → `computed()`

İhtiyacın HTTP isteği mi? → `resource()`

İhtiyacın side effect mi? (localStorage, DOM, log, 3rd party) → `effect()`

İhtiyacın signal'i set etmek mi? → Method veya event handler

❑ Bu Dersten Çıkarılacaklar

- ✓ `effect()`'in ne zaman tetiklendiğini detaylı biliyorum
- ✓ 5 use case'ini biliyorum (localStorage, title, analytics, 3rd party, debug)
- ✓ 3 anti-pattern'i tanıyorum (computed yerine, HTTP, signal set)
- ✓ cleanup mekanizmasını anlıyorum
- ✓ Karar ağacını uygulayabiliyorum

untracked() ve Tracking Kontrolü

Bazen effect veya computed içinde bir signal'i izlemek istemeyebilirsiniz. `untracked()` bu işe yarar.

❏ Senaryo

Diyelim ki effect'inde iki signal kullanıyorsun ama sadece birini izlemek istiyorsun. Diğeri sadece "anlık değer" olarak gerekli.

```
userId = signal('user-1');
lastLogTime = signal(Date.now());

constructor() {
  effect(() => {
    // userId değişince log atmak istiyorum
    console.log('User changed:', this.userId());

    // lastLogTime'ı OKUMAK istiyorum ama buna BAĞLI OLMAK istemiyorum
    console.log('Last log:', this.lastLogTime());

    // Eğer normal okursak, lastLogTime her güncellendiğinde effect yeniden çalışır
    // Ama biz sadece userId değişiminde çalışsın istiyoruz!
  });
}
```

Sorun: `lastLogTime` da izleniyor. Çözüm?

❏ Çözüm: `untracked()`

```
import { signal, effect, untracked } from '@angular/core';

userId = signal('user-1');
lastLogTime = signal(Date.now());

constructor() {
  effect(() => {
    // Bu izlenir
    console.log('User changed:', this.userId());

    // Bu OKUNur ama İZLENmez
    const time = untracked(() => this.lastLogTime());
    console.log('Last log:', time);
  });
}

// Şimdi:
this.userId.set('user-2');    // ✓ Effect tetiklenir
this.lastLogTime.set(Date.now()); // ✗ Effect tetiklenmez
```

▣ Pratik Use Case'ler

Use Case 1: Audit Logging

```
userActions = signal<Action[]>([]);
currentUser = signal<User | null>(null);

constructor() {
  effect(() => {
    const actions = this.userActions();

    // currentUser'ı sadece okumak istiyoruz, izlemek değil
    // Çünkü user değişince audit log yenilenmesin
    const user = untracked(() => this.currentUser());

    if (actions.length > 0 && user) {
      this.logToServer(user.id, actions);
    }
  });
}
```

Use Case 2: Computed'de One-Time Data

```
searchTerm = signal('');
debounceMs = signal(300); // Bu değişebilir ama tetiklemeli

debouncedSearch = computed(() => {
  // searchTerm değişince re-compute
  const term = this.searchTerm();

  // debounceMs sadece initial value'sı önemli, değişimi bizi etkilemiyor
  const ms = untracked(() => this.debounceMs());

  return { term, ms };
});
```

Use Case 3: Initialization

```
userPreferences = signal<Prefs>({...});
theme = signal('light');

constructor() {
  effect(() => {
    // theme değişimini izle
    const t = this.theme();

    // İlk preference değerini al, sonradan değişmesi bizi etkilemesin
    const prefs = untracked(() => this.userPreferences());

    this.applyTheme(t, prefs);
  });
}
```

⚠️ Önemli Notlar

- `untracked()` bir fonksiyon alır ve çalıştırır
- Bu fonksiyonun içinde okunan tüm `signal`'ler izlenmez
- `Effect/computed` dışında çağırırsan etkisi yok (zaten izleme yok)
- Aşırı kullanma — gerçekten gerekli değilse normal okuma daha açık

📦 Bu Dersten Çıkarılacaklar

- ✓ `untracked()`'in ne için olduğunu biliyorum
- ✓ `Effect` içinde "okumak ama izlememek" gerekliyse kullanabiliyorum
- ✓ 3 pratik use case'ini biliyorum
- ✓ Aşırı kullanmamam gerektiğini anlıyorum

linkedSignal() — Senkronize Signal

Angular 19 yeniliği. `linkedSignal()`, başka bir signal'a bağlı ama yine de yazılabilir bir signal yaratır.

❑ Hangi Problemi Çözüyor?

Senaryo: Bir dropdown'da seçili olan değer. Liste değişirse seçim sıfırlanmalı, ama kullanıcı manuel de değiştirebilmeli.

```
// ❑ Eski yöntem – manuel sync
options = signal<string[]>(['A', 'B', 'C']);
selected = signal<string>('A');

// Options değişince selected'ı sıfırla – manuel
updateOptions(newOptions: string[]) {
  this.options.set(newOptions);
  this.selected.set(newOptions[0]); // Manual reset
}
```

Sorun: Her options değişimi için manuel sync. Unutursan bug.

❑ Modern Çözüm: `linkedSignal()`

```
import { signal, linkedSignal } from '@angular/core';

options = signal<string[]>(['A', 'B', 'C']);

// linkedSignal – options değişince OTOMATİK reset
selected = linkedSignal(() => this.options()[0]);

// Otomatik:
this.options.set(['X', 'Y', 'Z']);
// selected şimdi 'X' (otomatik)

// Ama manuel set edebilirsiniz:
this.selected.set('Y');
// selected şimdi 'Y'

// Sonra options değişirse:
this.options.set(['A', 'B', 'C']);
// selected şimdi 'A' (yeniden reset)
```

❑ Use Cases

Use Case 1: Form Default Value

```
user = signal<User>({...});

// User değişince form default'ları yenile
formName = linkedSignal(() => this.user().name);
formEmail = linkedSignal(() => this.user().email);

// User değişirse form sıfırlanır
// Ama kullanıcı manuel de yazabilir
```

Use Case 2: Pagination Reset

```
filterTerm = signal('');
currentPage = linkedSignal(() => {
  // filterTerm okunması, ilk değer 1
  this.filterTerm(); // Just read to track
  return 1;
});

// filter değişirse page 1'e döner
// User manuel sayfa değiştirebilir
```

Use Case 3: Advanced Equality

```
// computation + equality + value access
import { linkedSignal } from '@angular/core';

selected = linkedSignal({
  source: () => this.items(),
  computation: (items, prev) => {
    // Önceki seçim hala listede mi?
    if (prev && items.find(i => i.id === prev.value.id)) {
      return prev.value; // Aynı seçimi koru
    }
    return items[0]; // Yoksa ilk item
  }
});
```

□ linkedSignal vs computed vs signal

u	computed	signal	linkedSignal
abilir?	Hayır	Evet	Evet
ka signal'a bağlı?	Evet	Hayır	Evet
uel override?	Yok	Var	Var (geçici)
ce değişince	—	Etkilenmez	Reset olur

□ Bu Dersten Çıkarılacaklar

- ✓ linkedSignal()'in çözdüğü problemi biliyorum
- ✓ Otomatik reset ve manual override mantığını anlıyorum

- ✓ Form default, pagination gibi senaryolarda kullanabiliyorum
- ✓ computed, signal, linkedSignal arasındaki farkları biliyorum

resource() ve httpResource()

Modern Angular'da async data yönetimi: resource() ve httpResource() (v19+). Promise/Observable'lardan kurtul.

❑ Eski Yol — Sorunlar

```
// ❑ Klasik approach
@Component({...})
export class UserComponent {
  userId = signal('123');
  user = signal<User | null>(null);
  loading = signal(false);
  error = signal<string | null>(null);

  constructor() {
    effect(() => {
      const id = this.userId();
      this.loading.set(true);
      this.error.set(null);

      this.http.get<User>(`/api/users/${id}`).subscribe({
        next: u => {
          this.user.set(u);
          this.loading.set(false);
        },
        error: e => {
          this.error.set(e.message);
          this.loading.set(false);
        }
      });
    });
  }
}
```

Problemler:

- ✗ 3 signal yönetmek gerek (data, loading, error)
- ✗ Manuel error handling
- ✗ Race condition riski (eski istek geç dönerse)
- ✗ Effect içinde HTTP isteği (anti-pattern)
- ✗ Cleanup yok (component destroy olursa istek devam eder)

❑ Modern Çözüm: resource()

```
import { signal, resource } from '@angular/core';

@Component({...})
export class UserComponent {
  userId = signal('123');

  // ÜÇÜ BİRDEN – data, loading, error
  userResource = resource({
    request: () => ({ id: this.userId() }),
    loader: async ({ request, abortSignal }) => {
      const response = await fetch(`/api/users/${request.id}`, {
        signal: abortSignal // Auto cancel
      });
      if (!response.ok) throw new Error('Failed');
      return response.json() as Promise<User>;
    },
  });

  // Kullanım:
  // userResource.value()    → User | undefined
  // userResource.isLoading() → boolean
  // userResource.error()    → unknown
  // userResource.status()   → 'idle' | 'loading' | 'resolved' | 'error'
}
```

□ Template Kullanımı

```
@switch (userResource.status()) {
  @case ('loading') {
    <app-spinner />
  }
  @case ('error') {
    <p>Hata: {{ userResource.error() }}</p>
    <button (click)="userResource.reload()">Tekrar Dene</button>
  }
  @case ('resolved') {
    @let user = userResource.value();
    <h1>{{ user.name }}</h1>
    <p>{{ user.email }}</p>
  }
}
```

⚡ resource'un Özellikleri

- ✓ Auto loading state: isLoading() otomatik yönetilir
- ✓ Auto error handling: error() ile hata yakalanır
- ✓ Auto cancellation: request değişince eski istek iptal
- ✓ Type-safe: request, value, error tipleri inferred
- ✓ Reactive request: request fonksiyonundaki signal'lar izlenir
- ✓ Cleanup: Component destroy'da otomatik

✓ Reload: resource.reload() ile manuel yenileme

□ **httpResource()** — Daha da Kolay

Angular v19+ ile gelen httpResource(): HttpClient ile entegre, daha az boilerplate.

```
import { httpResource } from '@angular/common/http';

@Component({...})
export class UserComponent {
  userId = signal('123');

  // Tek satır!
  userResource = httpResource<User>(() =>
    `/api/users/${this.userId()}`
  );
}

// Veya request config ile:
userResource = httpResource<User>(() => ({
  url: `/api/users/${this.userId()}`,
  method: 'GET',
  params: { include: 'profile' },
  headers: { 'X-Custom': 'value' },
})));
```

□ **Use Cases**

Use Case 1: Simple Fetch

```
users = httpResource<User[]>('/api/users');
// users.value() → User[] | undefined
```

Use Case 2: Search

```
query = signal('');

results = httpResource<Result[]>(() => {
  const q = this.query();
  if (!q) return undefined; // Skip request
  return `/api/search?q=${q}`;
});
```

Use Case 3: Pagination

```
page = signal(1);
pageSize = signal(20);

items = httpResource<PagedResult>(() => ({
  url: '/api/items',
  params: {
    page: this.page(),
    size: this.pageSize(),
  },
}));
```

□ Bu Dersten Çıkarılacaklar

- ✓ resource() ve httpResource() arasındaki farkı biliyorum
- ✓ Async data için bu API'ları kullanabilirim
- ✓ Auto loading/error/cancellation mekanizmalarını anlıyorum
- ✓ Effect içinde HTTP isteği yapmamam gerektiğini biliyorum
- ✓ Template'te status() ile farklı durumları handle edebilirim

RxJS Interop — toSignal() / toObservable()

Signal'lar harika ama RxJS hala çok güçlü. Bu ders ikisini birleştirmeyi öğretiyor.

☐ Köprü Fonksiyonları

```
import { toSignal, toObservable } from '@angular/core/rxjs-interop';
import { Observable } from 'rxjs';

// Observable → Signal
const obs$: Observable<number> = ...;
const sig = toSignal(obs$, { initialValue: 0 });

// Signal → Observable
const sig = signal(0);
const obs$ = toObservable(sig);
```

☐ toSignal() — En Yaygın Use Case

Use Case 1: HttpClient Observable'ı Signal'a Çevir

```
@Injectable({...})
export class UserService {
  http = inject(HttpClient);

  // Eski observable approach
  getUsers$ = this.http.get<User[]>('/api/users');

  // Modern signal approach
  users = toSignal(this.http.get<User[]>('/api/users'), {
    initialValue: [] as User[]
  });
}
```

Use Case 2: Router Events

```
router = inject(Router);

currentUrl = toSignal(
  this.router.events.pipe(
    filter(e => e instanceof NavigationEnd),
    map(e => (e as NavigationEnd).url)
  ),
  { initialValue: '/' }
);
```

Use Case 3: FormControl Value

```
titleControl = new FormControl('');

// Reactive form control'ün değerini signal olarak izle
title = toSignal(this.titleControl.valueChanges, {
  initialValue: ''
});

// Şimdi title() ile değeri okuyabilirsiniz
```

❑ toObservable() — Ters Yön

Signal'i Observable yap. RxJS operatörlerini kullanmak istediğinde:

```
import { debounceTime, switchMap } from 'rxjs/operators';

searchTerm = signal('');

// Signal'a debounce + switchMap uygula
searchResults$ = toObservable(this.searchTerm).pipe(
  debounceTime(300),
  switchMap(term => this.http.get(`/api/search?q=${term}`))
);

// Sonra istersen tekrar signal'a çevir
searchResults = toSignal(this.searchResults$, { initialValue: [] });
```

⚙️ toSignal() Options

```
toSignal(obs$, {
  initialValue: 0,           // İlk değer (önerilen)
  requireSync: false,       // true → ilk emit'i bekler (BehaviorSubject için)
  manualCleanup: false,     // true → manuel cleanup
  rejectErrors: false,      // true → error fırlatır, false → error signal'ı
});
```

❑ Hangi Durumda Hangisi?

Signal Kullan:

- ✓ Lokal component state
- ✓ Computed/derived values
- ✓ Form input değerleri
- ✓ UI state (modal açık mı, vs.)
- ✓ Async data — resource()/httpResource() ile

RxJS Observable Kullan:

- ✓ Complex async streams (debounce, throttle, switchMap)
- ✓ WebSocket connections
- ✓ Server-sent events
- ✓ Multi-step async operations
- ✓ Mevcut Observable-based library (3rd party)

☐ Bu Dersten Çıkarılacaklar

- ✓ toSignal() ile Observable'ı signal'a çevirebilirim
- ✓ toObservable() ile signal'i Observable'a çevirebilirim
- ✓ İki dünyayı birlikte kullanabilirim
- ✓ Hangi durumda hangisini seçeceğimi biliyorum

Modül 5 Özeti

Bu modülde Angular'ın kalbi olan reactivity sistemini derinlemesine inceledik. Artık modern Angular'da state yönetimi konusunda tam donanımlısin.

Neler Öğrendin?

- ✓ Signal nedir, neden var — Zone.js vs signals tarihsel bağlam
- ✓ signal() — yazılabilir kaynak, options, asReadonly pattern
- ✓ computed() — türetilmiş değer, memoization, glitch-free
- ✓ effect() — side effect, kurallar, anti-patterns
- ✓ untracked() — okumadan ama izlemeden
- ✓ linkedSignal() — bağlı ama yazılabilir
- ✓ resource() / httpResource() — modern async data
- ✓ RxJS interop — toSignal, toObservable

Anahtar Pattern'ler

1. Service'te State Yönetimi

```
@Injectable({ providedIn: 'root' })
export class DataService {
  // Private writable
  private _data = signal<Item[]>([]);

  // Public readonly
  readonly data = this._data.asReadonly();

  // Derived computed
  readonly count = computed(() => this._data().length);
  readonly hasItems = computed(() => this.count() > 0);

  // Actions
  add(item: Item) { this._data.update(arr => [...arr, item]); }
  remove(id: string) { this._data.update(arr => arr.filter(i => i.id !== id)); }
}
```

2. Async Data Pattern

```

@Component({...})
export class ListComponent {
  filter = signal('');

  items = httpResource<Item[]>(() => ({
    url: '/api/items',
    params: { filter: this.filter() }
  }));

  // Template: @switch (items.status()) { ... }
}

```

3. Side Effect Pattern

```

constructor() {
  // localStorage sync
  effect(() => {
    localStorage.setItem('state', JSON.stringify(this._state()));
  });

  // Analytics
  effect(() => {
    const route = this.currentRoute();
    if (route) gtag('event', 'page_view', { page: route });
  });
}

```

📦 Sıradaki Modül

Modül 6 — Lifecycle Hooks & Directives. Component yaşam döngüsü, lifecycle hook'lar, custom directive'ler ve structural directive'lerin modern alternatifleri.